Solar Power Generation Prediction for Indian States

A Two-Stage Classical Machine Learning Pipeline

Complete Project Documentation: From Zero to a Working Web Application

Random Forest · Gradient Boosting · Linear Regression

NASA POWER weather data · Plant generation data · Solar geometry

Prepared for a 15-day project plan

Version 1.0

Contents

1 Project Overview **4**

1.1 The problem . . . 4
1.2 The idea in one paragraph . . . 4
1.3 Objectives . . . 4
1.4 Scope: what is in and what is out . . . 5
1.5 Deliverables . . . 5
1.6 Success criteria . . . 5

2 Scientific Background **6**

2.1 Solar geometry . . . 6
2.1.1 Declination . . . 6
2.1.2 Hour angle . . . 6
2.1.3 Zenith and azimuth . . . 6
2.1.4 Angle of incidence on a tilted panel . . . 6
2.2 Solar radiation . . . 7
2.3 How a solar panel converts light to power . . . 7
2.3.1 Power depends on light . . . 7
2.3.2 Power drops when the panel is hot . . . 7
2.3.3 Voltage, current and power . . . 8
2.3.4 Series and parallel connection . . . 8
2.4 Weather effects on output . . . 8

3 Machine Learning Background **9**

3.1 Regression . . . 9
3.2 Train, test and why the split matters . . . 9
3.3 Metrics . . . 9
3.4 The baseline . . . 9
3.5 The three models . . . 10
3.5.1 Linear Regression . . . 10
3.5.2 Random Forest (bagging) . . . 10
3.5.3 Gradient Boosting . . . 10
3.5.4 Feature importance . . . 10

3.6 Overfitting and leakage . . . 10

4 System Design **11**
4.1 Architecture . . . 11
4.2 Why a bridge is needed . . . 11
4.3 The contract between the stages . . . 11
4.4 Main design decisions . . . 12

5 Data **13**
5.1 Dataset 1: NASA POWER daily weather (Stage 1) . . . 13
5.2 Dataset 2: plant generation data (Stage 2) . . . 13
5.3 State and union territory points . . . 14
5.4 Limitations of the data . . . 14

6 Data Collection and Merging **15**
6.1 Environment setup . . . 15
6.2 Project folder layout . . . 15
6.3 Downloading NASA data . . . 15
6.4 Troubleshooting the download . . . 16
6.5 Merging into one CSV . . . 16
6.6 What the merged file looks like . . . 16
6.7 Verifying the merged data . . . 16

7 Stage 1: Predicting Weather for a State and Period **18**
7.1 Goal . . . 18
7.2 Step 1: cleaning . . . 18
7.3 Step 2: dividing months into three periods (dekads) . . . 18
7.4 Step 3: features and targets . . . 19
7.5 Step 4: split by year . . . 19
7.6 Step 5: models . . . 19
7.7 Step 6: evaluation and what to expect . . . 19
7.8 Step 7: uncertainty . . . 19
7.9 Step 8: saving . . . 19
7.10 Analysis to include in your notebook . . . 20

8 Stage 2: Predicting Solar Power **21**
8.1 Goal . . . 21
8.2 Step 1: cleaning and daytime filter . . . 21
8.3 Step 2: verify the angle-of-incidence formula . . . 21
8.4 Step 3: features . . . 21
8.5 Step 4: target . . . 22
8.6 Step 5: split . . . 22

8.7 Step 6: models and tuning . . . 22
8.8 Step 7: evaluation . . . 22
8.9 Step 8: a sensitivity test . . . 23
8.10 Step 9: limits of validity . . . 23

9 Connecting the Stages (the Chain) 24
9.1 Step-by-step algorithm . . . 24
9.2 The calibration factor . . . 25
9.3 Error propagation . . . 25
9.4 Sanity checks that must pass . . . 25

10 The Web Application 26
10.1 Why Streamlit . . . 26
10.2 Page layout . . . 26
10.3 Running the app . . . 26
10.4 Deployment . . . 26
10.5 Testing checklist . . . 27

11 15-Day Roadmap 28
11.1 Weekly review questions . . . 29
11.2 If you run out of time . . . 29

12 README, Resume and Interview Preparation 30
12.1 README outline . . . 30
12.2 Resume bullets . . . 30
12.3 Interview questions and answers . . . 30
12.4 Limitations summary . . . 31
12.5 Future work . . . 31

A State and Union Territory Points 32

B Angle-of-Incidence Verification Code 34

C Project Code 35
C.1 common.py (shared helpers) . . . 35
C.2 stage1_train.py . . . 36
C.3 stage2_train.py . . . 37
C.4 chain.py . . . 38
C.5 app.py . . . 38

D Glossary 40

E Sources and Further Reading 41

Chapter 1

Project Overview

1.1 The problem

Solar energy output depends on the sun's position, the weather, the location and the way a panel is installed. A person who wants to install panels asks practical questions: *How much electricity will my panels generate in a given state of India on a given date? How much does it change if I tilt the panels differently or face them another way?*

This project builds a system that answers those questions using classical machine learning (no deep learning), 10 years of daily weather data for every Indian state and union territory, and a plant dataset with measured power output.

1.2 The idea in one paragraph

The user chooses a **state**, a **date**, a **panel tilt**, a **panel orientation**, the **number of panels** and the **rating of each panel**. **Stage 1** predicts the expected weather for that state and date: maximum temperature, minimum temperature, wind speed and solar radiation. The date is placed in one of 36 ten-day periods of the year (three per month). **Stage 2** takes the Stage 1 weather together with the sun's angle on the user's panel and predicts the solar power. The output of the first model is the input of the second model; that is the chain.

1.3 Objectives

1. Collect and clean 10 years of daily weather for 36 states and union territories.
2. Convert daily data into 10-day-period (dekad) statistics and train Stage 1 models.
3. Prepare the plant dataset and train Stage 2 models that predict power from weather and sun angle.
4. Connect both stages so Stage 1 predictions feed Stage 2.
5. Let the user change tilt, orientation, number of panels and panel rating.
6. Build and deploy a web application, and document everything honestly.

1.4 Scope: what is in and what is out

In scope	Out of scope
Classical ML: Linear Regression, Random Forest, Gradient Boosting	Deep learning, neural networks
Expected (typical) weather for a 10-day period	Day-ahead weather forecasting
One representative point per state	District-level or grid-level prediction
Fixed-tilt panels with user-chosen tilt and orientation	Tracking systems, shading analysis
Energy per day, peak power, panel voltage estimate	Detailed electrical design of inverters and cables

1.5 Deliverables

- A merged weather CSV (about 130,000 rows) and a cleaned plant dataset.
- Notebooks or scripts for preprocessing, analysis and training.
- Two trained model sets (Stage 1 and Stage 2) saved with `joblib`.
- A working web application (Streamlit) with source code on GitHub.
- A README, this documentation, and a short presentation or demo video.

1.6 Success criteria

- Stage 1 is compared against a historical-average baseline on years the models have not seen.
- Stage 2 is tested on the last 20% of the plant data (chronological split) and reports R^2, MAE and RMSE.
- The full chain gives sensible answers: more energy in clear seasons, less in the monsoon, a south-facing tilt beats a north-facing tilt in India.
- Every limitation is written down. An honest project with clear limits is stronger than one with unexplained claims.

Chapter 2

Scientific Background

You do not need to be a solar engineer, but you must understand why each input is in the model. This chapter explains the physics behind every feature.

2.1 Solar geometry

2.1.1 Declination

The earth's axis is tilted, so the sun's latitude on the earth (declination δ) changes through the year between about $-23.45°$ and $+23.45°$:

$$\delta = 23.45° \sin\left(\frac{360°}{365}(284 + n)\right) \tag{2.1}$$

where n is the day of the year (1 to 365).

2.1.2 Hour angle

The hour angle ω measures how far the sun is from its noon position. It is $0°$ at solar noon and changes by $15°$ every hour:

$$\omega = 15° \, (t_{\text{solar}} - 12) \tag{2.2}$$

It is negative in the morning and positive in the afternoon.

2.1.3 Zenith and azimuth

For a location at latitude φ, the sun's **zenith angle** θ_z (angle from straight overhead) satisfies

$$\cos\theta_z = \sin\varphi \sin\delta + \cos\varphi \cos\delta \cos\omega \tag{2.3}$$

The sun is above the horizon when $\theta_z < 90°$. The **solar azimuth** γ_s is the compass direction of the sun. In this project azimuth is measured **from north, clockwise**, so east is $90°$, south is $180°$ and west is $270°$. This is the same convention as the plant dataset.

2.1.4 Angle of incidence on a tilted panel

A panel has a **tilt** β (angle from flat; $0°$ is lying flat, $90°$ is vertical) and an **orientation** γ (the compass direction it faces). The angle of incidence θ is the angle between the sun's rays and the panel's normal:

$$\cos\theta = \cos\theta_z \cos\beta + \sin\theta_z \sin\beta \cos(\gamma_s - \gamma) \tag{2.4}$$

Direct sunlight on the panel is proportional to $\cos\theta$. When $\cos\theta \leq 0$ the sun is behind the panel and direct light is zero. In India (northern hemisphere) a fixed panel normally faces south, $\gamma = 180°$, with a tilt close to the latitude.

Note. On the sample plant rows we fitted β and γ to the column `angle_of_incidence` using this formula. The best fit was a tilt of about 45° facing south (180°) with an error of 0.07°. The formula and the azimuth convention above therefore match the dataset exactly. You must repeat this check on the full file (code in Appendix B).

2.2 Solar radiation

- **Extraterrestrial radiation** (H_0) is the energy reaching the top of the atmosphere on a horizontal surface. It depends only on latitude and date. It is a clean, physical feature that tells the model how much sun is available before clouds and air reduce it. The daily value in kWh/m^2/day is

$$H_0 = \frac{24}{\pi} G_{sc} \left(1 + 0.033 \cos\frac{2\pi n}{365}\right) (\cos\varphi \cos\delta \sin\omega_s + \omega_s \sin\varphi \sin\delta) \tag{2.5}$$

  with $G_{sc} = 1.367$ kW/m^2 and sunset hour angle $\omega_s = \arccos(-\tan\varphi \tan\delta)$ (in radians).

- **Global horizontal irradiance (GHI)** is the sunlight arriving on a flat horizontal surface at ground level. NASA POWER's `ALLSKY_SFC_SW_DWN` is a daily GHI in kWh/m^2/day. The plant dataset has hourly GHI-like values in W/m^2 (`shortwave_radiation_backwards_sfc`, an average over the previous hour).

- **Clear-sky versus all-sky:** clouds, humidity and dust reduce the clear-sky value. The monsoon months are the clearest example in India.

2.3 How a solar panel converts light to power

2.3.1 Power depends on light

Current is almost proportional to the light on the panel; so power is almost proportional to irradiance G (W/m^2) at a fixed temperature.

2.3.2 Power drops when the panel is hot

Cell temperature T_c is higher than air temperature. A common estimate uses the nominal operating cell temperature (NOCT):

$$T_c = T_a + \frac{G}{800}(\text{NOCT} - 20) \tag{2.6}$$

Power then changes by roughly -0.35 to -0.45% per °C above 25°C:

$$P = P_{\text{STC}} \frac{G}{1000} \left[1 + \gamma\,(T_c - 25)\right], \quad \gamma \approx -0.004 \text{ per °C} \tag{2.7}$$

Wind cools the panel and lowers T_c, so wind has a small positive effect on output.

2.3.3 Voltage, current and power

The voltage at maximum power (V_{mp}) is mostly decided by panel design (the number of cells) and falls about 0.3% per °C. It changes only slowly with sunlight (logarithmically). Current follows sunlight. So **power and current vary a lot during the day, voltage does not**. A common approximation for one panel with N_s cells is

$$V_{mp} \approx V_{mp,\mathrm{STC}}\,[1 + \beta_V (T_c - 25)] + N_s\, n\, V_T \ln \frac{G}{1000}, \qquad V_T = \frac{kT}{q} \tag{2.8}$$

For a 72-cell, 400 Wp reference module we use $V_{mp,\mathrm{STC}} = 41$ V, $\beta_V = -0.0028$ per °C and $n = 1.2$. Voltage is not present in the plant dataset, so this project computes it from physics, not machine learning, and says so.

2.3.4 Series and parallel connection

Panels in a string add their voltages; strings in parallel add their currents:

$$V_{\mathrm{array}} = N_{\mathrm{series}}\, V_{mp}, \qquad I_{\mathrm{array}} = N_{\mathrm{strings}}\, I_{mp}, \qquad P_{\mathrm{array}} = V_{\mathrm{array}}\, I_{\mathrm{array}} \tag{2.9}$$

2.4 Weather effects on output

Factor	Effect	Model input
Sun height	Low sun means a long path through air and less energy	Zenith (inside AOI)
Angle of incidence	Output scales with $\cos\theta$	`aoi`, `cos_aoi`
Radiation	Almost proportional to power	`ghi`
Temperature	About -0.4% per °C above 25°C	`temp`
Wind	Cools the panel slightly	`wind`
Cloud cover	Cuts direct light, leaves diffuse light	Inside radiation
Humidity, dust	Scatter light and add soiling	Inside radiation
System losses	Inverter, wiring, shading, ageing (10 to 15%)	Already in plant output

Chapter 3

Machine Learning Background

3.1 Regression

Regression predicts a number (temperature, power) from input features. The model learns a function $\hat{y} = f(x)$ from examples where both x and y are known.

3.2 Train, test and why the split matters

You fit the model on a **training set** and judge it on a **test set** it has never seen. If you judge a model on data it trained on, you only measure memory.

Important. Random splits are wrong for time-ordered data. Neighbouring hours and neighbouring days are almost identical, so a random split lets the model peek at the answer through near-duplicates. This project always splits by time: Stage 1 by **year**, Stage 2 by **row order** (first 80% train, last 20% test).

3.3 Metrics

With true values y_i, predictions $\hat{y}_i$ and n samples:

$$\text{MAE} = \frac{1}{n}\sum_i |y_i - \hat{y}_i| \qquad \text{(average error in the original unit)} \tag{3.1}$$

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_i (y_i - \hat{y}_i)^2} \qquad \text{(punishes big errors more)} \tag{3.2}$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \qquad \text{(1 is perfect, 0 equals always predicting the mean)} \tag{3.3}$$

For Stage 1 we report MAE in °C, m/s and kWh/m^2/day because those units are easy to explain. For Stage 2 we report R^2, MAE and RMSE in kW.

3.4 The baseline

A baseline is the simplest sensible prediction. For Stage 1 it is the **climatology baseline**: the average of the same state and the same 10-day period over the training years. A machine learning model is only worth using if it beats its baseline on unseen data. For Stage 2 the baseline is Linear Regression.

3.5 The three models

3.5.1 Linear Regression

Fits $\hat{y} = w_0 + w_1 x_1 + \cdots + w_k x_k$. It is fast and easy to explain but cannot represent curves or interactions unless you engineer features. It is the baseline for Stage 2.

3.5.2 Random Forest (bagging)

A Random Forest builds many decision trees, each on a random sample of the rows and a random subset of the features at every split, then averages their predictions. Averaging many different trees **reduces variance** (the model's sensitivity to the particular training sample). Important settings: `n_estimators` (number of trees), `max_depth`, `min_samples_leaf`. Trees cannot extrapolate beyond the range of training targets and features. This matters for tilt angles later in the project.

3.5.3 Gradient Boosting

Gradient Boosting builds shallow trees one after another; each new tree fits the errors (residuals) left by the previous ones, scaled by a learning rate. Boosting mainly **reduces bias** (underfitting). Important settings: `n_estimators`, `learning_rate`, `max_depth` (usually 2 to 4). A small learning rate with more trees usually generalises better.

3.5.4 Feature importance

Tree models report how much each feature reduced the error. Permutation importance (shuffle one feature and measure how much the score drops) is a more trustworthy alternative. Always include a feature importance chart in the README; it shows the model learned physics, for example that `g_cos` (radiation times $\cos\theta$) dominates.

3.6 Overfitting and leakage

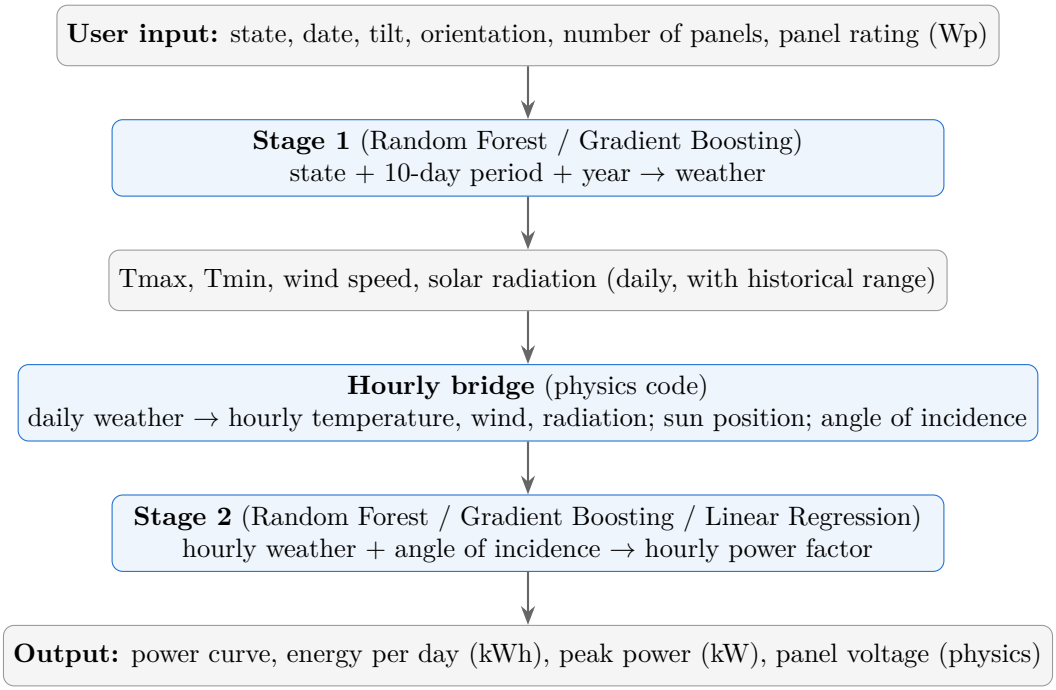
Overfitting means doing well on training data and badly on new data. Control it with a held-out test set, limited tree depth and leaf size, and fewer features. **Leakage** means information about the answer reaching the model through the inputs. Examples in this project:

- Random splits on time series (explained above).
- Using columns that identify the fixed panel geometry (the plant's zenith and azimuth) so the model cannot respond to a new tilt (see Chapter 8).
- Training Stage 2 on one source of weather and testing the chain on another without checking they agree.

Chapter 4

System Design

4.1 Architecture



4.2 Why a bridge is needed

Stage 1 produces *daily* values (maximum temperature, minimum temperature, daily radiation). Stage 2 is trained on *hourly* rows (temperature at that hour, radiation at that hour, sun angle at that hour). The bridge turns daily numbers into a believable hourly day, so Stage 2 receives inputs that look like its training data. The bridge uses simple physics and no learning (Chapter 9). The timestamps exist only inside this code; the plant dataset does not need them for training.

4.3 The contract between the stages

The second model may only use inputs that the first model can supply or that code can compute. This rule decides which plant columns are kept.

Source	Allowed Stage 2 inputs
From Stage 1	Temperature, wind speed (10 m), solar radiation
Computed by code	Angle of incidence (from location, date, hour, tilt, orientation) and features derived from it
Dropped	Pressure, snowfall, precipitation, wind direction, wind gust, wind at 80 m and 900 mb, cloud layers (Stage 1 cannot predict them)
Dropped on purpose	Plant zenith and azimuth columns (they identify the fixed panel and would stop the model from reacting to a new tilt)

Note. If you want humidity or cloud cover in Stage 2, add NASA POWER parameters `RH2M` and `CLOUD_AMT` to the download and to Stage 1 targets. The core version in this document keeps the model small and explainable.

4.4 Main design decisions

Decision	Reason
10-day periods (dekads)	Matches the idea: three equal-ish parts per month; reduces daily noise
Climatology baseline	Shows honestly whether ML adds value
Split by year / by order	Prevents the future from leaking into training
Power as a capacity factor	Makes output scalable to any number of panels and any rating
Voltage from physics	Voltage is not in any dataset; it is mostly set by panel design
Show historical range	Stage 1 predicts typical weather, not tomorrow's weather

Chapter 5

Data

5.1 Dataset 1: NASA POWER daily weather (Stage 1)

NASA POWER (Prediction Of Worldwide Energy Resources) provides free, keyless access to daily and hourly weather and solar data from satellite and reanalysis sources, through a web API. We request one point per state and union territory.

Column in our CSV	NASA parameter	Unit
`tmax`	`T2M_MAX`	°C
`tmin`	`T2M_MIN`	°C
`wind_speed`	`WS10M`	m/s
`solar_radiation`	`ALLSKY_SFC_SW_DWN`	kWh/m^2/day

Request details: endpoint `https://power.larc.nasa.gov/api/temporal/daily/point`, community `RE`, one request per state and year, JSON format. The reply marks missing values as `-999`. The units were confirmed in the reply header during a test request. The time standard is local standard time.

Time span. The project uses 2016 to 2025 (10 years). NASA's solar data starts around 1981, so a longer record is possible later, but 10 years is enough to finish in 15 days. Ten years gives a limited view of long-term warming; say so in your report.

5.2 Dataset 2: plant generation data (Stage 2)

The plant dataset contains hourly rows with weather, sun angles and measured power (`generated_power_kw`). The sample has 4,214 rows and 21 columns.

Column	Use	Comment
temperature_2_m_above_gnd	Keep	Air temperature, °C
wind_speed_10_m_above_gnd	Keep	Matches NASA `WS10M`
shortwave_radiation_backwards_sfc	Keep	GHI, W/m^2, mean of previous hour
angle_of_incidence	Keep	Sun-to-panel angle, degrees
zenith, azimuth	Verify only	Used to check the AOI formula; not model inputs
generated_power_kw	**Target**	Measured power
relative_humidity, pressure, precipitation, snowfall	Drop	Not supplied by Stage 1
cloud cover (total, high, medium, low)	Drop	Not supplied by Stage 1
wind direction, wind at 80 m / 900 mb, wind gust	Drop	Not supplied by Stage 1

5.3 State and union territory points

One representative point per state is used (Appendix A). NASA's grid cells are about 50 km wide, so each value is a rough average for that area. Large or varied states (Rajasthan, Maharashtra, Jammu and Kashmir) hide differences inside them. Adding more points per state is a future improvement.

5.4 Limitations of the data

- The plant is probably not in India. Its relationship between radiation, temperature, angle of incidence and power is physical and transfers well, but the exact output belongs to "a plant like this one".
- The plant has one fixed tilt (about 45°, south). Other tilts are handled only through the angle of incidence.
- NASA and the plant dataset come from different weather sources. Compare their radiation and temperature distributions before trusting the chain.
- The plant's true nameplate capacity is unknown. Chapter 9 explains how this is handled with a calibration factor.

Chapter 6

Data Collection and Merging

6.1 Environment setup

1. Install Python 3.10 or newer from python.org (tick "Add Python to PATH").
2. Install VS Code (or any editor) and Git.
3. Create the project folder and a virtual environment:

```
cd "Solar Panel Power Generator Prediction"
python -m venv venv
venv\Scripts\activate
pip install requests pandas numpy scikit-learn joblib matplotlib seaborn streamlit
```

6.2 Project folder layout

```
Solar Panel Power Generator Prediction/
  nasa_raw/                    one folder per state, one CSV per year
  data/
    nasa_power_india_daily.csv merged weather (about 130,000 rows)
    plant_data.csv             plant dataset with generated_power_kw
  common.py                    shared helper functions
  stage1_train.py              trains Stage 1
  stage2_train.py              trains Stage 2
  chain.py                     connects the stages
  app.py                       Streamlit web app
  notebooks/                   EDA notebooks (optional)
  requirements.txt
  README.md
```

6.3 Downloading NASA data

The downloader requests 36 states $\times$ 10 years = 360 small files and saves each as it arrives. It tries three different download methods in random order, skips failures and repeats passes until nothing is missing. You can stop it with Ctrl+C and run it again; it continues where it stopped.

```
python nasa_power_downloader.py
```

Each pass prints how many chunks are still missing. A normal run has a few failed chunks in pass 1 and fewer in each later pass. Pass 1 takes roughly 15 to 30 minutes.

6.4 Troubleshooting the download

Problem	What to do
Connection reset (error 10054)	Random server or network reset. Keep the script running; it retries. Try a mobile hotspot, turn off VPN or proxy, pause the antivirus web shield
Browser test works but Python fails	Run the script from another terminal (Command Prompt instead of VS Code, or the reverse)
State has fewer rows than the others	Some years are missing. Run the downloader again
Empty state folder	The folder is created at the start; wait for the downloads to finish
Script runs but "No CSV files found" during merge	Wrong folder. Run from the project folder or pass the path of `nasa_raw`

6.5 Merging into one CSV

The merge script reads every CSV below `nasa_raw`, converts `-999` to blank, removes duplicates, sorts by state and date, and writes one file.

```
python merge_nasa_csv.py
python merge_nasa_csv.py "C:\path\to\nasa_raw"      (if the folder is elsewhere)
```

Keep the merge script **outside** the `nasa_raw` folder; otherwise it will not find the data folder.

6.6 What the merged file looks like

```
state,lat,lon,date,tmax,tmin,wind_speed,solar_radiation
Karnataka,15.32,75.71,2025-01-01,30.52,16.57,4.49,4.6418
Karnataka,15.32,75.71,2025-01-02,29.42,15.01,4.67,3.8882
Rajasthan,27.02,74.22,2025-01-01,...
```

(The Karnataka values come from a real test response of the API for a point in Karnataka.)

6.7 Verifying the merged data

Run these checks before any modelling:

```
import pandas as pd
df = pd.read_csv("nasa_power_india_daily.csv", parse_dates=["date"])
print(df.shape)                                  # about (131500, 8)
print(df.groupby("state").size().describe())     # 3653 for every state (2016-2025)
print(df.isna().sum())                           # missing values per column
print(df.describe().T)                           # min, max, mean per column
```

Expected: 36 states, 3,653 days each, very few missing values. Temperatures roughly -30 to $50°C$, wind speed 0 to 15 m/s, radiation 0 to about 8 kWh/m^2/day. Anything outside these ranges needs investigation.

Chapter 7

Stage 1: Predicting Weather for a State and Period

7.1 Goal

Given a state and a date, predict the expected maximum temperature, minimum temperature, wind speed and solar radiation for the 10-day period containing that date.

7.2 Step 1: cleaning

1. Convert `-999` to missing (the merge script does this).
2. Sort by state and date.
3. Fill short gaps (up to 5 days) by linear interpolation inside each state; drop what remains.
4. Check ranges and look for impossible values ($T_{\min} > T_{\max}$, negative radiation).

7.3 Step 2: dividing months into three periods (dekads)

Each month is split into three parts: days 1 to 10, days 11 to 20, and day 21 to the end of the month. This gives 36 periods per year. Because months have 28 to 31 days, the last period has 8 to 11 days, so the periods are not exactly equal. Index 0 is 1–10 January and index 35 is 21–31 December:

$$\text{dk} = (\text{month} - 1) \times 3 + \begin{cases} 0 & \text{day} \le 10 \\ 1 & 11 \le \text{day} \le 20 \\ 2 & \text{day} \ge 21 \end{cases} \tag{7.1}$$

Example. 17 October 2026: month = 10, day = 17, so $\text{dk} = 9 \times 3 + 1 = 28$ (the 29th period, 11–20 October).

Each period's value is the mean of the daily values inside it. The result is one row per **state, year, period**: $36 \times 10 \times 36 = 12{,}960$ rows.

7.4 Step 3: features and targets

Feature	Meaning
`lat`, `lon`	Location of the state point
`dk`	Period of the year, 0 to 35
`dk_sin`, `dk_cos`	Cyclical encoding so period 35 sits next to period 0
`year`	Captures the trend over years
`h0`	Extraterrestrial radiation for that latitude and period (physics feature)
Targets	`tmax`, `tmin`, `wind_speed`, `solar_radiation`

Trees cannot extrapolate. With only 10 years, a model predicting for 2026 repeats the last years' level and misses part of the warming trend. Mention this in the limitations.

7.5 Step 4: split by year

Train on 2016 to 2022 and test on 2023 to 2025. Never use a random split.

7.6 Step 5: models

1. **Climatology baseline:** the training-year average for the same state and period.
2. **Random Forest:** 300 trees, `min_samples_leaf=2`, multi-output (all four targets at once).
3. **Gradient Boosting:** 300 trees, learning rate 0.05, depth 3, one model per target through `MultiOutputRegressor`.

The code is in Appendix C (`stage1_train.py`).

7.7 Step 6: evaluation and what to expect

Report the MAE of each model for each target in the units of the target, in a table. A typical honest finding for period-level prediction is that the models are **close to the baseline**. A period model learns the typical weather plus a trend, and it cannot know whether next week will be cloudy. This is a valid result. Report it as "ML matches or slightly improves the climatology baseline on unseen years". Do not hide the baseline.

7.8 Step 7: uncertainty

For each state and period, store the 10th and 90th percentile of the historical values (`history_ranges.csv`). The app shows "expected value (typical range)" instead of a single number.

7.9 Step 8: saving

Save the trained models with `joblib` and keep the helper tables: `states.csv` (state, latitude, longitude) and `history_ranges.csv`.

7.10 Analysis to include in your notebook

- Seasonal curves by state: Tmax, Tmin and radiation across the 36 periods.
- Monsoon dip in radiation and cooling in Tmax for central, eastern and southern states.
- Correlation heatmap of Tmax, Tmin, wind and radiation.
- Box plots of each target by state.
- Predicted versus actual for the test years.

Chapter 8

Stage 2: Predicting Solar Power

8.1 Goal

Given hourly temperature, wind speed, solar radiation and the angle of incidence on the panel, predict the power the plant produces.

8.2 Step 1: cleaning and daytime filter

1. Load the plant CSV and keep the original row order (do not shuffle).
2. Drop rows with missing values in the used columns.
3. Keep only rows with zenith $< 90°$. Rows with the sun below the horizon have meaningless angles of incidence and add nothing.
4. Check for unrealistic values: negative power, radiation above about 1,200 W/m^2, or temperatures outside the climate of the site.

8.3 Step 2: verify the angle-of-incidence formula

Fit the panel's tilt β and azimuth γ to the dataset's `angle_of_incidence`, `zenith` and `azimuth` columns using Equation (2.4) (code in Appendix B). On the 16 daytime sample rows the fit gave $\beta \approx 45°$, $\gamma \approx 180°$ with a root-mean-square error of 0.07°. If the full file gives an error below about 1°, you have proved that the angle you compute for a user's panel is on the same footing as the one in the data. Report the fitted tilt and azimuth in your README.

8.4 Step 3: features

The core feature set is deliberately small:

Feature	Meaning
`temp`	Air temperature (°C)
`wind`	Wind speed at 10 m (m/s)
`ghi`	Solar radiation (W/m^2)
`aoi`	Angle of incidence (degrees)
`cos_aoi`	$\max(\cos\theta, 0)$
`g_cos`	`ghi` × `cos_aoi`, a proxy for the light that hits the panel directly

Important. Do **not** give Stage 2 the plant's zenith and azimuth as inputs. For a fixed panel, zenith and azimuth together determine the angle of incidence, so a tree model can learn the panel's geometry from them and ignore `aoi`. Then changing the tilt in the app would have little effect. Keeping only `aoi` (and features made from it) forces the model to react to the angle you compute for the user's panel.

8.5 Step 4: target

The target is the power as a fraction of a reference output:

$$\text{cf} = \frac{P_{\text{plant}}}{P_{\text{ref}}}, \qquad P_{\text{ref}} = 99.5\text{th percentile of the plant's power} \tag{8.1}$$

The model predicts cf. The final output for any system is cf × (number of panels × W_p/1000) in kW. This is how one plant's data scales to any number of panels. Chapter 9 explains the accuracy limit of this step.

8.6 Step 5: split

Use the first 80% of rows (in time order) for training and the last 20% for testing. Never use a random split.

8.7 Step 6: models and tuning

Train three models and compare them:

1. **Linear Regression** (baseline).
2. **Random Forest:** 300 trees, `min_samples_leaf=3`.
3. **Gradient Boosting:** 400 trees, learning rate 0.05, depth 3.

For tuning, use `TimeSeriesSplit` (not ordinary K-fold) with `RandomizedSearchCV` over tree count, depth, leaf size and learning rate, on the training part only. Test only once at the end.

8.8 Step 7: evaluation

Report R^2, MAE and RMSE in kW for each model on the test set. Add these figures:

- Predicted versus actual power (scatter with a diagonal line).
- Predicted and actual power over time for one test week.

- Feature importance of the best tree model.
- Residuals versus radiation and versus angle of incidence.

Because clear-sky hours dominate the data, the models usually reach a high R^2. Interpret it carefully and also report MAE in kW.

8.9 Step 8: a sensitivity test

Before building the app, test that the model reacts sensibly. For one fixed hour of weather, change only the angle of incidence from 0° to 90° and plot the predicted power. It should fall smoothly toward zero. Trees give a staircase shape, which is acceptable. A flat line means the model ignores the angle and the feature design needs fixing.

8.10 Step 9: limits of validity

- The training panel is fixed at one tilt. Predictions are trustworthy for angles of incidence inside the range seen in training (roughly 15° to 90° in the sample).
- Diffuse light and ground reflection depend on tilt too; the model only sees them through the plant's single tilt.
- Radiation in the plant data and in NASA POWER come from different sources. Compare them.

Chapter 9

Connecting the Stages (the Chain)

9.1 Step-by-step algorithm

For a user's state, date, tilt β, orientation γ, number of panels N and panel rating W_p:

1. **Stage 1.** Look up the state's latitude and longitude. Build the features (period index, year, H_0, location) and predict $T_{\max}$, $T_{\min}$, wind $\bar{w}$ and daily radiation H (kWh/m^2/day). Enforce $T_{\max} > T_{\min}$, wind ≥ 0, $H \geq 0$.
2. **Hourly time axis.** Use the middle of each hour, $t = 0.5, 1.5, \ldots, 23.5$ (solar time).
3. **Sun position.** Compute θ_z and γ_s for each hour from latitude and day of year.
4. **Hourly radiation.** Weight each daylight hour by $\cos\theta_z$ (sun height) and scale so the day adds up to the Stage 1 daily total:

$$G_h = 1000\, H\, \frac{\cos\theta_{z,h}}{\sum_k \cos\theta_{z,k}} \quad \text{(W/m}^2\text{ for hours with the sun up)} \tag{9.1}$$

   Summing G_h over 24 hours of one hour each gives $1000\,H$ Wh/m^2, which is the daily total.
5. **Hourly temperature.** A smooth curve between the minimum and the maximum, with its peak at 15:00:

$$T_h = T_{\min} + (T_{\max} - T_{\min}) \left[0.5 + 0.5 \sin \frac{\pi(t-9)}{12}\right] \tag{9.2}$$

6. **Wind.** Constant at the daily mean.
7. **Angle of incidence.** Use the user's β and γ in Equation (2.4) for each hour. If $\theta \geq 90°$ the sun is behind the panel; set power to zero.
8. **Stage 2.** Build the Stage 2 features for each hour and predict the capacity factor cf.
9. **Scale to the system.** Power in kW is cf $\times\, NW_p/1000 \times c$, where c is the calibration factor (below).
10. **Daily energy.** Add the 24 hourly powers (each is an average over one hour, so kW sums to kWh). Peak power is the largest hour.
11. **Voltage.** Compute the panel's V_{mp} for each hour from the physics formula; show the peak-hour value and the string voltage $N_{\text{series}} V_{mp}$.

9.2 The calibration factor

Step 4 of Stage 2 normalised power by the plant's 99.5th percentile output, not by its true nameplate rating, which is unknown. A real plant at its peak usually produces less than its nameplate rating, so using this normalisation tends to **overestimate** the output of a system with the given rating by 10 to 20%. Fix it with an honest calibration:

1. Pick 3 to 5 states and one fixed tilt and orientation.
2. Get hourly simulated PV output for the same points from PVGIS (a free European Commission tool and API) for 1 kWp.
3. Compute the ratio of PVGIS yearly or monthly energy to the chain's energy and use the median ratio as c.

State clearly in your README that c was calibrated against PVGIS. Until you calibrate, keep $c = 1.0$ and label results as "relative, uncalibrated".

9.3 Error propagation

Stage 1 errors flow into Stage 2. Measure this in three tests:

1. **Stage 2 alone** on the held-out plant rows (real inputs).
2. **Bridge test:** take days from the plant data, compute daily Tmax, Tmin, wind and radiation, rebuild the hourly day with the bridge, run Stage 2 and compare daily energy with the measured energy. This isolates the loss caused by the daily-to-hourly step. Days can be separated by finding where the azimuth wraps back to its morning values.
3. **Full chain versus PVGIS:** for 3 to 5 states and several dates, compare the chain's daily energy with PVGIS for the same tilt and orientation.

Present the three results in one table. Expect each step to add some error; report them without hiding any.

9.4 Sanity checks that must pass

- Energy in clear months (March to May) is higher than in monsoon months for most states.
- A south-facing panel (orientation 180°) at a tilt near the latitude beats a north-facing panel (orientation 0°) in India.
- A flat panel ($\beta = 0°$) gives more energy than a steep one in June and less in December for northern states.
- Doubling the number of panels exactly doubles the energy.

Chapter 10

The Web Application

10.1 Why Streamlit

Streamlit turns a Python script into a web page. It needs no HTML or JavaScript and deploys free on Streamlit Community Cloud. It runs your saved models directly, which is simpler than building a separate API.

10.2 Page layout

Area	Content
Inputs	State, date, number of panels, panel rating (Wp), tilt slider, orientation slider (180 = south)
Stage 1 cards	Expected Tmax, Tmin, wind, radiation, each with the historical range
Stage 2 cards	Energy per day, peak power, panel voltage at peak, string voltage
Chart	Hourly power curve
Model page	Metrics tables, feature importance, predicted-versus-actual charts
About page	Data sources, method, limitations

10.3 Running the app

```
python stage1_train.py
python stage2_train.py
streamlit run app.py
```

Order matters: both training scripts create the model files that the app loads. The full code is in Appendix C.

10.4 Deployment

1. Put the project on GitHub (include the model files if they are small, or generate them in a startup script).

2. Create `requirements.txt`:

```
streamlit
pandas
numpy
scikit-learn
joblib
```

3. On share.streamlit.io choose your repository and `app.py` as the main file, then deploy.

Keep the `scikit-learn` version the same as the one you trained with; models saved with one version may not load in a different one. Pin versions in `requirements.txt` (for example `scikit-learn==1.8.0`).

10.5 Testing checklist

- Every state in the list works for several dates throughout the year.
- Changing a slider changes the output.
- Leap day (29 February) does not crash the app.
- Very large and very small panel counts give proportional output.
- The page loads in a reasonable time (models cached with `st.cache_resource`).

Chapter 11

15-Day Roadmap

Day	Goal	Tasks	Deliverable
1	Foundations	Read Chapters 2–3. Set up Python, VS Code, Git and the virtual environment. Create the GitHub repository	Repo with folder layout and README stub
2	Data download	Run the NASA downloader. Check progress per pass. Fix connection problems	`nasa_raw` folders with CSVs
3	Merge and verify	Merge into one CSV. Run the verification checks. Inspect ranges and missing values	`nasa_power_india_daily.csv`, data report
4	Plant data audit	Load the plant CSV. Inspect columns, ranges, order. Run the AOI fit to verify tilt and azimuth	Notes on plant data, fitted tilt and azimuth
5	EDA (weather)	Seasonal curves, monsoon effect, correlation heatmap, box plots by state	EDA notebook with 6–8 charts
6	Stage 1 data prep	Interpolate gaps, build dekads, add `h0`, write the dekad table	`dekad_table.csv`
7	Stage 1 modelling	Baseline, Random Forest, Gradient Boosting. Split by year. Metrics table	Stage 1 models and results table
8	Stage 2 data prep	Daytime filter, features, capacity-factor target, chronological split	Clean Stage 2 dataset
9	Stage 2 modelling	Linear Regression, Random Forest, Gradient Boosting. Tuning with `TimeSeriesSplit`	Stage 2 model and metrics
10	Stage 2 analysis	Feature importance, residuals, sensitivity test on angle of incidence	Analysis notebook
11	Chain	Write `common.py` and `chain.py`. Run the sanity checks	Working chain function
12	Validation	Bridge test. PVGIS comparison for a few states. Choose calibration factor	Validation table
13	App	Build `app.py`. Test all states and sliders	Local web app

Day	Goal	Tasks	Deliverable
14	Deploy	`requirements.txt`, GitHub, Streamlit Cloud. Record a short demo	Public link
15	Polish	Final README, limitations, resume bullets, practice explaining each choice	Finished project and presentation

11.1 Weekly review questions

- Can I explain every column in my model and why it is there?
- Did every split respect time order?
- Does my model beat its baseline, and by how much?
- Which limitation would an interviewer find first, and have I already written it down?

11.2 If you run out of time

Cut in this order: (1) extra charts, (2) the PVGIS calibration (keep $c = 1$ and label results as uncalibrated), (3) the bridge test. Never cut the baseline comparison, the time-based splits or the limitations section.

Chapter 12

README, Resume and Interview Preparation

12.1 README outline

1. Title, one-sentence description, screenshot or link to the live app.
2. Problem and approach (two-stage chain, diagram).
3. Data sources and how to obtain them (NASA POWER, plant dataset).
4. How to run (install, train, launch).
5. Results: Stage 1 table with baseline, Stage 2 table, chain validation table.
6. Limitations and assumptions (list from this document).
7. Future work.

12.2 Resume bullets

Replace the placeholders with your real numbers.

- Built a two-stage ML system that predicts solar power for any Indian state and date: stage 1 forecasts expected weather (temperature, wind, radiation) from 10 years of NASA POWER data across 36 regions; stage 2 maps weather and sun angle to power using Random Forest and Gradient Boosting (R^2 = X on a time-based test set).
- Engineered physics features (extraterrestrial radiation, angle of incidence) and compared models with a climatology baseline using year-based and chronological splits to avoid leakage.
- Deployed a Streamlit app where users set tilt, orientation, panel count and rating to get hourly power, daily energy and panel voltage.

12.3 Interview questions and answers

Q: Why split by year and not randomly? Weather and power in adjacent days and hours are nearly identical. A random split lets the model see near-copies of test rows, so scores look better than real performance.

Q: Why did Stage 1 barely beat the baseline? A 10-day period model can only learn the typical weather and trend, not day-to-day variation. The average of past years is already a strong predictor, and I reported that honestly.

Q: How does tilt affect your model if the plant has one tilt? I recompute the angle of incidence for the user's tilt and orientation with the standard solar geometry formula, which I verified on the dataset (error 0.07° on the sample). The model sees tilt only through that angle.

Q: Why did you drop zenith and azimuth from Stage 2? They identify the plant's fixed panel; with them the model could ignore the angle of incidence and not react to a new tilt.

Q: Why is voltage not predicted by machine learning? No dataset has voltage, and voltage is mostly determined by panel design and temperature. I used the standard formula and labelled it as physics.

Q: Why Random Forest versus Gradient Boosting? Random Forest averages independent trees (lower variance); Gradient Boosting adds trees that correct earlier errors (lower bias). I compared both and chose by test error.

Q: What would you improve with more time? More points per state, 50 years of data from a reanalysis source, extra inputs such as humidity and cloud cover, several plants with different tilts, and PVGIS-based calibration.

12.4 Limitations summary

- Stage 1 predicts typical conditions, not tomorrow's weather.
- One point per state hides internal variation.
- Only 10 years of weather; trees cannot extrapolate the warming trend.
- The plant is probably outside India with one tilt; output scaling depends on a calibration factor.
- Radiation comes from two different sources.
- Voltage is a physics estimate, not a learned prediction.

12.5 Future work

District-level predictions, longer records, more plants and tilts, humidity and cloud inputs, uncertainty intervals for power, and a tracking-panel option.

Appendix A

State and Union Territory Points

One approximate central point per state or union territory is used for the NASA POWER download.

#	State / union territory	Latitude	Longitude
1	Andhra Pradesh	15.91	79.74
2	Arunachal Pradesh	28.22	94.73
3	Assam	26.20	92.94
4	Bihar	25.10	85.31
5	Chhattisgarh	21.28	81.87
6	Goa	15.30	74.12
7	Gujarat	22.26	71.19
8	Haryana	29.06	76.09
9	Himachal Pradesh	31.10	77.17
10	Jharkhand	23.61	85.28
11	Karnataka	15.32	75.71
12	Kerala	10.85	76.27
13	Madhya Pradesh	22.97	78.66
14	Maharashtra	19.75	75.71
15	Manipur	24.66	93.91
16	Meghalaya	25.47	91.37
17	Mizoram	23.16	92.94
18	Nagaland	26.16	94.56
19	Odisha	20.95	85.10
20	Punjab	31.15	75.34
21	Rajasthan	27.02	74.22
22	Sikkim	27.53	88.51
23	Tamil Nadu	11.13	78.66
24	Telangana	18.11	79.02
25	Tripura	23.94	91.99
26	Uttar Pradesh	26.85	80.91
27	Uttarakhand	30.07	79.02

#	State / union territory	Latitude	Longitude
28	West Bengal	22.99	87.85
29	Delhi	28.61	77.21
30	Jammu and Kashmir	33.78	76.58
31	Ladakh	34.15	77.58
32	Puducherry	11.94	79.81
33	Chandigarh	30.73	76.78
34	Andaman and Nicobar	11.74	92.66
35	Lakshadweep	10.57	72.64
36	Dadra Nagar Haveli and Daman Diu	20.40	72.83

Appendix B

Angle-of-Incidence Verification Code

This script fits the plant panel's tilt and azimuth to the `angle_of_incidence` column. Install SciPy first (`pip install scipy`).

```
import numpy as np, pandas as pd
from scipy.optimize import least_squares

df = pd.read_csv("plant_data.csv")
df = df[df["zenith"] < 90]                                  # daytime rows only
z, a, aoi = [np.radians(df[c].values) for c in ("zenith", "azimuth", "angle_of_incidence")]

def model_aoi(p):                                           # p = (tilt, panel azimuth) in degrees
    b, g = np.radians(p)
    c = np.cos(z) * np.cos(b) + np.sin(z) * np.sin(b) * np.cos(a - g)
    return np.arccos(np.clip(c, -1, 1))

r = least_squares(lambda p: model_aoi(p) - aoi, [40, 180])
rmse = np.degrees(np.sqrt(np.mean((model_aoi(r.x) - aoi) ** 2)))
print("tilt, azimuth:", r.x, " RMSE (degrees):", rmse)   # sample rows gave about 45, 180, 0.07
```

Appendix C

Project Code

All files live in the project folder. Run order: `stage1_train.py`, `stage2_train.py`, then `streamlit run app.py`. The files were tested on synthetic data that mimics the real files; run them on your real data and read every printed result.

C.1 common.py (shared helpers)

```python
"""Shared helper functions (used by training scripts and by the app)."""
import numpy as np
import pandas as pd

D = np.pi / 180
MD = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]


def dekad_index(month, day):
    """0..35: three periods per month (1-10, 11-20, 21-end)."""
    month, day = np.asarray(month), np.asarray(day)
    return (month - 1) * 3 + np.where(day <= 10, 0, np.where(day <= 20, 1, 2))


def _dekad_mid_doy():
    mids, start = [], 0
    for m in range(12):
        for a, b in [(1, 10), (11, 20), (21, MD[m])]:
            mids.append(start + (a + b) / 2)
        start += MD[m]
    return np.array(mids)


DEKAD_MID_DOY = _dekad_mid_doy()


def h0_daily(lat, doy):
    """Daily extraterrestrial radiation on a horizontal surface, kWh/m2/day."""
    lat, doy = np.asarray(lat, float), np.asarray(doy, float)
    dec = 23.45 * D * np.sin(2 * np.pi * (284 + doy) / 365)
    p = lat * D
    ws = np.arccos(np.clip(-np.tan(p) * np.tan(dec), -1, 1))
    return (24 / np.pi * 1.367 * (1 + 0.033 * np.cos(2 * np.pi * doy / 365)) *
            (np.cos(p) * np.cos(dec) * np.sin(ws) + ws * np.sin(p) * np.sin(dec)))


def stage1_features(lat, lon, month, day, year):
    lat, lon, month, day, year = np.broadcast_arrays(*[np.atleast_1d(v) for v in (lat, lon, month, day,
     year)])
    dk = dekad_index(month, day)
    return pd.DataFrame({
        "lat": lat, "lon": lon, "dk": dk,
        "dk_sin": np.sin(2 * np.pi * dk / 36), "dk_cos": np.cos(2 * np.pi * dk / 36),
        "year": year, "h0": h0_daily(lat, DEKAD_MID_DOY[dk]),
```

```
    })


def sun_position(lat, doy, hour):
    """Zenith and azimuth in degrees. Azimuth from north, clockwise (180 = south).
    hour = local solar time, 0..24."""
    dec = 23.45 * D * np.sin(2 * np.pi * (284 + doy) / 365)
    om = 15 * D * (np.asarray(hour, float) - 12)
    p = lat * D
    cz = np.clip(np.sin(p) * np.sin(dec) + np.cos(p) * np.cos(dec) * np.cos(om), -1, 1)
    z = np.arccos(cz)
    sz = np.maximum(np.sin(z), 1e-6)
    az = np.arccos(np.clip((np.sin(dec) - cz * np.sin(p)) / (sz * np.cos(p)), -1, 1))
    az = np.where(om > 0, 2 * np.pi - az, az)
    return np.degrees(z), np.degrees(az)


def aoi_deg(zenith, sun_az, tilt, panel_az):
    """Angle between sun rays and the panel normal, degrees."""
    z, a, b, g = [np.radians(np.asarray(v, float)) for v in (zenith, sun_az, tilt, panel_az)]
    c = np.cos(z) * np.cos(b) + np.sin(z) * np.sin(b) * np.cos(a - g)
    return np.degrees(np.arccos(np.clip(c, -1, 1)))


S2_FEATS = ["temp", "wind", "ghi", "aoi", "cos_aoi", "g_cos"]


def stage2_features(temp, wind, ghi, aoi):
    c = np.clip(np.cos(np.radians(np.asarray(aoi, float))), 0, None)
    return pd.DataFrame({"temp": np.asarray(temp, float), "wind": np.asarray(wind, float),
                         "ghi": np.asarray(ghi, float), "aoi": np.asarray(aoi, float),
                         "cos_aoi": c, "g_cos": np.asarray(ghi, float) * c})
```

C.2 stage1_train.py

```
"""Stage 1: predict tmax, tmin, wind_speed, solar_radiation for (state, 10-day period)."""
import joblib
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.multioutput import MultiOutputRegressor
from sklearn.metrics import mean_absolute_error
from common import dekad_index, h0_daily, DEKAD_MID_DOY

TARGETS = ["tmax", "tmin", "wind_speed", "solar_radiation"]
FEATS = ["lat", "lon", "dk", "dk_sin", "dk_cos", "year", "h0"]
TRAIN_END = 2022            # train on years <= 2022, test on later years

# 1) load and clean the daily data
df = pd.read_csv("nasa_power_india_daily.csv", parse_dates=["date"])
df = df.sort_values(["state", "date"]).reset_index(drop=True)
df[TARGETS] = df.groupby("state")[TARGETS].transform(
    lambda s: s.interpolate(limit=5, limit_direction="both"))
df = df.dropna(subset=TARGETS)
df["year"] = df["date"].dt.year
df["dk"] = dekad_index(df["date"].dt.month, df["date"].dt.day)

# 2) daily -> 10-day means (one row per state, year, period)
tab = (df.groupby(["state", "lat", "lon", "year", "dk"], as_index=False)[TARGETS].mean())
tab = tab.reset_index(drop=True)
dk = tab["dk"].values
tab["dk_sin"] = np.sin(2 * np.pi * dk / 36)
tab["dk_cos"] = np.cos(2 * np.pi * dk / 36)
tab["h0"] = h0_daily(tab["lat"].values, DEKAD_MID_DOY[dk])
tab.to_csv("dekad_table.csv", index=False)

# 3) split by YEAR (never random: the future must not leak into training)
train, test = tab[tab["year"] <= TRAIN_END], tab[tab["year"] > TRAIN_END]
print("train rows", len(train), "test rows", len(test))

```

```
# 4) baseline = historical mean of the same state and period
clim = train.groupby(["state", "dk"])[TARGETS].mean().rename(columns=lambda c: c + "_clim")
base = test.join(clim, on=["state", "dk"])

models = {
    "RandomForest": RandomForestRegressor(n_estimators=300, min_samples_leaf=2, n_jobs=-1, random_state
     =42),
    "GradientBoosting": MultiOutputRegressor(GradientBoostingRegressor(
        n_estimators=300, learning_rate=0.05, max_depth=3, random_state=42)),
}
rows = [["Baseline"] + [mean_absolute_error(test[t], base[t + "_clim"]) for t in TARGETS]]
for name, m in models.items():
    m.fit(train[FEATS], train[TARGETS])
    pred = m.predict(test[FEATS])
    rows.append([name] + [mean_absolute_error(test[t], pred[:, i]) for i, t in enumerate(TARGETS)])
    joblib.dump(m, f"stage1_{name}.joblib")
print(pd.DataFrame(rows, columns=["model"] + TARGETS).round(3).to_string(index=False))

# 5) save helper tables for the app (state list and historical ranges)
tab[["state", "lat", "lon"]].drop_duplicates().to_csv("states.csv", index=False)
hist = tab.groupby(["state", "dk"])[TARGETS].quantile([0.1, 0.9]).unstack()
hist.columns = [f"{a}_p{int(b*100)}" for a, b in hist.columns]
hist.reset_index().to_csv("history_ranges.csv", index=False)
print("saved: dekad_table.csv, states.csv, history_ranges.csv, stage1_*.joblib")
```

C.3 stage2_train.py

Set `PLANT_CSV` to the name of your plant file.

```
"""Stage 2: hourly weather + angle of incidence -> power (as a capacity factor)."""
import joblib
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
from common import stage2_features, S2_FEATS

PLANT_CSV = "plant_data.csv"        # the file with generated_power_kw
df = pd.read_csv(PLANT_CSV)
need = ["temperature_2_m_above_gnd", "wind_speed_10_m_above_gnd", "shortwave_radiation_backwards_sfc",
        "angle_of_incidence", "zenith", "generated_power_kw"]
df = df.dropna(subset=need)
df = df[df["zenith"] < 90].reset_index(drop=True)          # daytime rows only, keep time order

X = stage2_features(df["temperature_2_m_above_gnd"], df["wind_speed_10_m_above_gnd"],
                    df["shortwave_radiation_backwards_sfc"], df["angle_of_incidence"])[S2_FEATS]
pref = df["generated_power_kw"].quantile(0.995)             # reference output of this plant
y = (df["generated_power_kw"] / pref).clip(0, 1.2)

cut = int(len(df) * 0.8)                                    # first 80% train, last 20% test
Xtr, Xte, ytr, yte = X[:cut], X[cut:], y[:cut], y[cut:]

models = {
    "LinearRegression": LinearRegression(),
    "RandomForest": RandomForestRegressor(n_estimators=300, min_samples_leaf=3, n_jobs=-1, random_state
     =42),
    "GradientBoosting": GradientBoostingRegressor(n_estimators=400, learning_rate=0.05, max_depth=3,
     random_state=42),
}
best, best_r2 = None, -9
for name, m in models.items():
    m.fit(Xtr, ytr)
    p = np.clip(m.predict(Xte), 0, None)
    r2 = r2_score(yte, p)
    print(f"{name:18s} R2={r2:.4f}  MAE={mean_absolute_error(yte, p) * pref:.1f} kW  "
          f"RMSE={mean_squared_error(yte, p) ** 0.5 * pref:.1f} kW")
    if r2 > best_r2:
        best, best_r2, best_name = m, r2, name
print("best:", best_name)
```

```
if hasattr(best, "feature_importances_"):
    print(dict(zip(S2_FEATS, best.feature_importances_.round(3))))
joblib.dump({"model": best, "pref": float(pref), "features": S2_FEATS}, "stage2_model.joblib")
```

C.4 chain.py

```
"""Chain: Stage 1 (daily weather) -> hourly profile -> Stage 2 (power) -> daily energy."""
import numpy as np
import pandas as pd
from common import sun_position, aoi_deg, stage1_features, stage2_features


def hourly_profile(tmax, tmin, wind, rad_kwh, lat, doy):
    hours = np.arange(0.5, 24, 1.0)                          # middle of each hour, solar time
    zen, azi = sun_position(lat, doy, hours)
    w = np.where(zen < 90, np.cos(np.radians(zen)), 0.0)     # sun height weights
    ghi = rad_kwh * 1000 * w / max(w.sum(), 1e-9)            # W/m2, average for each hour
    temp = tmin + (tmax - tmin) * (0.5 + 0.5 * np.sin(np.pi * (hours - 9) / 12))
    return pd.DataFrame({"hour": hours, "zenith": zen, "azimuth": azi,
                         "ghi": ghi, "temp": temp, "wind": wind})


def panel_vmp(g_poa, t_amb, vmp_stc=41.0, beta=-0.0028, cells=72):
    """Voltage at max power of one panel (physics formula, no ML)."""
    g_poa = np.maximum(np.asarray(g_poa, float), 1.0)
    tc = t_amb + g_poa / 800 * 25
    vt = 0.02569 * (tc + 273.15) / 298.15
    return np.maximum(0, vmp_stc * (1 + beta * (tc - 25)) + cells * 1.2 * vt * np.log(g_poa / 1000))


def predict_day(s1_model, s2, lat, lon, date, tilt, panel_az, n_panels, wp, calibration=1.0):
    d = pd.Timestamp(date)
    tmax, tmin, wind, rad = [float(v) for v in
                             s1_model.predict(stage1_features(lat, lon, d.month, d.day, d.year))[0]]
    tmax, wind, rad = max(tmax, tmin + 1.0), max(wind, 0.0), max(rad, 0.0)
    prof = hourly_profile(tmax, tmin, wind, rad, lat, d.dayofyear)
    prof["aoi"] = aoi_deg(prof["zenith"], prof["azimuth"], tilt, panel_az)
    X2 = stage2_features(prof["temp"], prof["wind"], prof["ghi"], prof["aoi"])[s2["features"]]
    cf = np.clip(s2["model"].predict(X2), 0, None)
    sunlit = (prof["zenith"] < 90) & (prof["aoi"] < 90) & (prof["ghi"] > 0)
    kwp = n_panels * wp / 1000
    prof["power_kw"] = np.where(sunlit, cf, 0.0) * kwp * calibration
    g_poa = prof["ghi"] * np.clip(np.cos(np.radians(prof["aoi"])), 0, None) / np.maximum(
        np.cos(np.radians(prof["zenith"])), 0.1)
    prof["vmp"] = np.where(sunlit, panel_vmp(g_poa, prof["temp"]), 0.0)
    return {"tmax": tmax, "tmin": tmin, "wind": wind, "radiation": rad,
            "energy_kwh": float(prof["power_kw"].sum()), "peak_kw": float(prof["power_kw"].max()),
            "profile": prof}
```

C.5 app.py

```
"""Streamlit app. Run:  streamlit run app.py"""
import datetime as dt
import joblib
import pandas as pd
import streamlit as st
from chain import predict_day

st.set_page_config(page_title="Solar Power Predictor - India", layout="wide")
st.title("Solar Power Prediction for Indian States")


@st.cache_resource
def load():
    return (pd.read_csv("states.csv"), pd.read_csv("history_ranges.csv"),
            joblib.load("stage1_RandomForest.joblib"), joblib.load("stage2_model.joblib"))
```

```
16
17
18  states, hist, s1, s2 = load()
19  c1, c2, c3 = st.columns(3)
20  state = c1.selectbox("State / UT", states["state"])
21  date = c2.date_input("Date", dt.date.today() + dt.timedelta(days=1))
22  n = c3.number_input("Number of panels", 1, 10000, 20)
23  c4, c5, c6 = st.columns(3)
24  wp = c4.number_input("Panel rating (Wp)", 50, 800, 400)
25  tilt = c5.slider("Tilt (degrees)", 0, 90, 25)
26  az = c6.slider("Orientation (180 = south)", 0, 359, 180)
27
28  row = states[states["state"] == state].iloc[0]
29  r = predict_day(s1, s2, row["lat"], row["lon"], date, tilt, az, n, wp)
30
31  st.subheader("Stage 1: expected weather for this 10-day period")
32  a, b, c, d = st.columns(4)
33  a.metric("Max temp (C)", f"{r['tmax']:.1f}")
34  b.metric("Min temp (C)", f"{r['tmin']:.1f}")
35  c.metric("Wind (m/s)", f"{r['wind']:.1f}")
36  d.metric("Solar radiation (kWh/m2/day)", f"{r['radiation']:.2f}")
37
38  st.subheader("Stage 2: solar output")
39  a, b, c = st.columns(3)
40  a.metric("Energy for the day (kWh)", f"{r['energy_kwh']:.1f}")
41  b.metric("Peak power (kW)", f"{r['peak_kw']:.2f}")
42  c.metric("Panel Vmp at peak (V)", f"{r['profile']['vmp'].max():.1f}")
43  st.line_chart(r["profile"].set_index("hour")["power_kw"])
44  st.caption("Typical conditions for the period, not a day-ahead weather forecast.")
```

Appendix D

Glossary

AOI	Angle of incidence: the angle between sunlight and the panel's normal.
Azimuth	Compass direction (from north, clockwise) of the sun or the panel.
Baseline	A simple prediction a model must beat to be useful.
Capacity factor (cf)	Output divided by a reference output; used to scale to any system size.
Dekad	A 10-day period; here, each month split into days 1–10, 11–20, 21–end.
GHI	Global horizontal irradiance: sunlight on a flat horizontal surface.
Leakage	Information about the answer reaching the model through inputs or splits.
NOCT	Nominal operating cell temperature, a panel rating used to estimate cell temperature.
Tilt	Angle of the panel from horizontal.
Vmp	Voltage of a panel at its maximum power point.
Wp	Watt-peak: panel rating under standard test conditions.
Zenith	Angle of the sun from straight overhead.

Appendix E

Sources and Further Reading

- NASA POWER project and API documentation: `https://power.larc.nasa.gov` (parameter dictionary and API pages).
- PVGIS (European Commission Joint Research Centre), photovoltaic performance tool: `https://re.jrc.ec.europa.eu/pvg_tools/en/`.
- IMD gridded daily temperature (1° resolution from 1951) through the `imdlib` Python package; useful if you later extend Stage 1 to 50 years.
- pvlib-python documentation: `https://pvlib-python.readthedocs.io`.
- scikit-learn documentation: `https://scikit-learn.org`.
- J. A. Duffie and W. A. Beckman, *Solar Engineering of Thermal Processes*, Wiley (solar geometry and radiation equations).
- L. Breiman, "Random Forests", *Machine Learning* 45, 2001.
- J. H. Friedman, "Greedy function approximation: a gradient boosting machine", *Annals of Statistics* 29(5), 2001.

Always check parameter names, units and start years in the official documentation before relying on them; services change over time.